

# Enabling Semantic Analysis Between Software Artifacts and Architectural Hypotheses Through AI Agents (Hypo-stage)

Research Proposal - MAC0499

Author: Fernando Drekner de Souza

Advisor: Prof. Eduardo Guerra

Co-advisor: José Gonçalves Lima Neto

Co-advisor: Prof. Paulo Roberto Miranda Meirelles

May 6, 2026

## 1 Introduction and motivation

Modern day Software Engineering faces the challenge of satisfying both agile development frameworks – in order to achieve desired results more effectively – and also a well-structured architecture, which would provide such software the possibility of being, for example, modular and easily testable and expandable. However, these demands eventually collide, as the necessity of planning whole structures in advance goes against the requirements of the frameworks previously mentioned [3].

Taking that into account, aligned with the clustered knowledge that revolves around Software Architecture, three fronts emerged to work together aiming to offer engineers the possibility of addressing said opposing ideas – Hypotheses Engineering introduced the very first concepts on handling and testing architectural ambiguities; Architectural Hypotheses presented an abstract fundamental arrangement to the first one; And Hypo-stage is the tool which brought these fundamentals to a tangible fruition [1] [2] [3].

Nevertheless, while still being a powerful tool to create and manage such hypotheses, Hypo-stage currently faces the challenge of integrating them with software artifacts that might surge in real time, for instance, user stories, tasks and their respective outcomes. This results in a lack of traceability of the interaction between architectural decisions and software requirements.

In light of this, and as AI surfaces as the latest trend in the Software Engineering scope [4], this project proposes the addition of AI agents to Hypo-stage as a helper tool for its users, with the purpose of providing a constant analysis of the inbound requisites and concatenate them with new features in order to evolve hypotheses and reduce uncertainty. Additionally, a successful implementation of such agents would also mean that a more cohesive and scalable architecture was achieved due to the reduction of uncertainties, as well as a reduced learning curve, independently of the user's tool usage experience.

## 2 Methodologies

The main idea of this research is contributing to an open source project with AI agents which will assist engineers in several fronts – Primarily, these agents will operate integrating hypotheses with the software engineering artifacts, as stated in the previous section. To do so, it will be necessary for them to assimilate how software requirements impact on said hypotheses. This brings a more philosophical approach to the agents' behavior and, as a result, will reduce redundancies, together with more associative hypotheses, which means a more harmonious architecture. Also, from a more technical standpoint, the agents will be able to understand how new improvements to the code result in hypotheses evolution in execution time. This would deliver better adjustments to hypotheses' uncertainties and impact, no to mention the detection of possible invalidity of such. Data to enhance such analysis could be retrieved from logs, pull requests and metrics, for instance.

Consequently, when both of these approaches are working gracefully, user experience would be improved, as there would be a quicker understanding of key concepts regarding the topic. This would represent more ease to make a great use of the Hypo-stage tool and would influence more users to adopt it to their software.

Furthermore, as to evaluate if the desired agents’ impact was achieved, there would be a study case analyzing their performance within real projects. Such study would assess both usages of the Hypo-stage tool with and without the agents. The metrics to understand their value would be both philosophical and numerical, such as time to create new hypotheses, their quality and cohesion, uncertainty reduction and lowering of the learning curve.

### 3 Objectives and Expected Results

By the end of the year, the project’s success will be determined by the effectiveness of the AI agents in supporting the integration between architectural hypotheses and software engineering artifacts, as well as their impact on reducing uncertainty and improving traceability, which will be measured by the following objectives:

- The agents are capable of continuously analyzing software artifacts, such as user stories, tasks, pull requests and execution logs, and establishing a significant connection between these artifacts and either existing or newly created hypotheses, improving architectural traceability.
- The use of the agents results in considerable improvements when in comparison with the tool without the assistance of such agents, including:
  - Reduction in the time required to create and evolve hypotheses;
  - Improved cohesion and relevance of hypotheses;
  - Reduction of uncertainty levels associated with architectural decisions.
- The agents are able to suggest new hypotheses and refinements to existing ones based on both real-time execution results and inbound requirements, contributing to a more adaptive and scalable software architecture.
- Users, either newcomers or more experienced, express a demonstrate a lower learning curve when interacting with Hypo-stage, supported by quicker task completion and improved understanding of Hypotheses Engineering core concepts.
- A real-world case study demonstrates that the integration of AI agents leads to a more consistent alignment between architectural decisions and software evolution over time.

Once these objectives have been met, it would ensure that AI agents can effectively operate not only as generative tools focused on code, but also as active participants in architectural planning. This would validate their role in supporting decision-making processes in software development, particularly in contexts that involve uncertainty, evolution and interaction with real-time software requirements.

### 4 Chronological Plan

|                                    | Mar | Apr | May | Jun | Jul | Aug | Sep | Oct | Nov | Dec |
|------------------------------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| Reference studying                 | X   | X   | X   |     |     |     |     |     |     |     |
| Building the agents                |     | X   | X   | X   | X   |     |     |     |     |     |
| Deploying the agents to production |     |     |     |     | X   | X   | X   | X   |     |     |
| Analyzing results                  |     |     |     |     |     | X   | X   | X   | X   |     |
| Paper writing                      |     |     |     |     | X   | X   | X   | X   | X   |     |
| Poster design                      |     |     |     |     |     |     |     |     | X   |     |
| Final presentation                 |     |     |     |     |     |     |     |     |     | X   |

### References

- [1] E. Guerra et al. “ArchHypo: Managing Software Architecture Uncertainty Using Hypotheses Engineering”. In: *IEEE Transactions on Software Engineering*, vol. 51, no. 2. Institute of Electrical and Electronics Engineers (IEEE), 2025.
- [2] E. Guerra et al. “Using Hypotheses to Manage Technical Uncertainty and Architecture Evolution in a Software Start-up”. In: *IEEE Software*, vol. 41, no. 4. Institute of Electrical and Electronics Engineers (IEEE), 2024.

- [3] Pedro Henrique Mariano Corrêa. “ArchHypo Tooling - Enabling Practical Hypothesis Engineering for Software Architecture”. Capstone Project. Universidade de São Paulo, 2025.
- [4] Seyedmoein Mohsenimofidi et al. *Context Engineering for AI Agents in Open-Source Software*. 2026. arXiv: 2510.21413 [cs.SE]. URL: <https://arxiv.org/abs/2510.21413>.